Chapter 01: Program Structure and Variables

ENG3091

Khiem Nguyen

Contents

1 Your first C++ program **2**
- 1.1 Hello, world! . . . 2
- 1.2 From source code to a running program . . . 2
- 1.3 Statements . . . 2
- 1.4 Syntax and syntax errors . . . 3

2 Comments **3**
- 2.1 Single-line comments . . . 3
- 2.2 Multi-line comments . . . 4
- 2.3 Comment *why*, not *what* . . . 4

3 Objects and variables **4**
- 3.1 Objects and variables . . . 4
- 3.2 Defining a variable . . . 5
- 3.3 A few fundamental types . . . 5
- 3.4 Naming variables . . . 5

4 Assignment and initialization **6**
- 4.1 Assignment . . . 6
- 4.2 Initialization . . . 6
- 4.3 Five ways to initialize . . . 6
- 4.4 Why braces? They stop narrowing . . . 7
- 4.5 Unused variables . . . 7

5 Input and output with `iostream` **8**
- 5.1 The `iostream` library . . . 8
- 5.2 Printing with `std::cout` . . . 8
- 5.3 New lines: `'\n'` or `std::endl`? . . . 9
- 5.4 Reading input with `std::cin` . . . 9
- 5.5 `<<` and `>>`: follow the arrows . . . 10
- 5.6 `std::cin` is buffered . . . 10
- 5.7 How the input buffer works . . . 11

6 Uninitialized variables and undefined behaviour **11**
- 6.1 Uninitialized variables . . . 11
- 6.2 What does an uninitialized variable hold? . . . 11
- 6.3 Garbage values in action . . . 12
- 6.4 Same code, different answers . . . 12
- 6.5 Undefined behaviour . . . 13

7 Wrapping up **13**
- 7.1 Summary . . . 13

1 Your first C++ program

1.1 Hello, world!

Listing 1 `hello.cpp`

```cpp
#include <iostream> // gives us std::cout

int main() {                        // every program starts here
    std::cout << "Hello world!\n"; // a statement ends with ;
    return 0;                     // tell the OS: all went well
}
```

```
Hello world!
```

Every C++ program you'll ever write has this shape:

- `#include <iostream>` brings in the code we need to print text (more on this later today).
- `int main() { ... }` is the **main function**. Every C++ program must have exactly one, and it's where the program starts.
- Between the braces `{` and `}` are **statements**, which run in order, from top to bottom.
- `return 0;` tells the operating system that everything went fine.

1.2 From source code to a running program




```
$ g++ -std=c++20 -Wall -Wextra hello.cpp -o hello
$ ./hello
Hello world!
```

- `g++` turns your source code into a program; `-o hello` names it. (On Windows, the program is `hello.exe`.)
- `-std=c++20` picks the version of C++; `-Wall -Wextra` turns on the useful warnings. Always use them!
- Your IDE runs these commands for you when you press "Run". Watch me do both in the lecture.

1.3 Statements

- When we tell another person something, we use sentences. When we tell a computer what to do, we write **statements**.
- Statements are by far the most common instruction in a C++ program.
- Most statements end with a **semicolon** `;`. Forgetting it is the classic beginner's mistake.

You'll meet many kinds of statements during the course, for example:

Kind	Example	Where we meet it
Declaration	`int x;`	today
Expression	`x = 5;`	today
Selection	`if (x > 0) { ... }`	Chapter 03
Iteration (loops)	`while (x > 0) { ... }`	Chapter 03
Jump	`return 0;`	today and Chapter 02

1.4 Syntax and syntax errors

English has grammar; C++ has **syntax**: rules about how a program must be written. The compiler checks them, and if you break one you get a **syntax error** and no program.

Listing 2 `hello_error.cpp`

```cpp
#include <iostream>

int main() {
    std::cout << "Hello world!\n" // oops: forgot the ;
        return 0;
}
```

```
hello_error.cpp:4:34: error: expected ';' before 'return'
```

`4:34` means line 4, column 34: exactly where the `;` should go. g++ noticed when it found `return` on the next line, hence "before 'return'". Other compilers may point at line 5 instead, so check the line it points to *and the one before*.

> **Best practice**
>
> Fix the **first** error the compiler reports, then compile again. One mistake often causes a whole list of follow-on errors.

2 Comments

2.1 Single-line comments

`//` starts a comment: the compiler ignores everything from `//` to the end of the line.

```cpp
std::cout << "Hello world!\n"; // everything from here on is ignored

// A comment on its own line keeps long lines short.
std::cout << "It is very nice to meet you!\n";
```

> **Coming from Python**
>
> Python comments start with `#`. In C++ they start with `//`. Careful: in C++, a line starting with `#` is something else entirely, an instruction to the *preprocessor*, like `#include <iostream>`.

2.2 Multi-line comments

Everything between `/*` and `*/` is ignored, however many lines it spans.

```
/* This is a multi-line comment.
   This line is ignored.
   So is this one. */

/* Some programmers line up stars
 * on the left, which makes long
 * comments easier to read.
 */
```

> **Watch out**
>
> Multi-line comments can't be nested: `/* outer /* inner */ still outer? */` ends at the **first** `*/`, and the compiler then chokes on `still outer? */`.

2.3 Comment *why*, not *what*

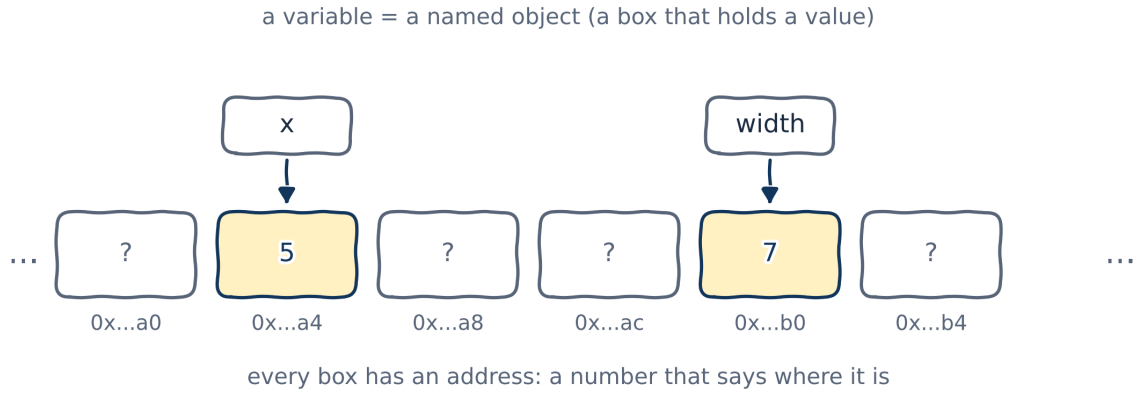
The code already says *what* it does. A good comment says *why*.

```
cost = cost * 1.2; // multiply cost by 1.2   (useless: the code says that)
cost = cost * 1.2; // add 20% VAT            (useful: now we know why)
```

> **Best practice**
>
> Write comments for the person who reads your code next. Very often, that person is you in six months.

3 Objects and variables

3.1 Objects and variables



- A program stores values in memory. C++ lets us use memory through **objects**: an object is a region of memory that can hold a value.
- An object can have a name, called an **identifier**. A named object is a **variable**.
- The word "object" means much more in object-oriented programming, which is where this course is heading.

3.2 Defining a variable

To get a variable, we write a **definition**: a type, then a name, then a semicolon.

```cpp
int x; // define a variable named x, of type int (a whole number)
```

Listing 3 `define_variables.cpp`

```cpp
int main() {
    int x; // define a variable named x, of type int
    int y; // define another one, named y

    int a, b; // two variables in one statement: note the comma
    // int c, int d;  // wrong: this is a compile error

    // Better: one definition per line, easier to read
    int total;
    double average;
    return 0;
}
```

This compiles, but with `-Wall` g++ warns that none of these variables is ever used. Fair enough: it's only a demo!

3.3 A few fundamental types

```cpp
int count;     // whole numbers:            -3, 0, 42
double price;  // numbers with a fraction:  3.99, -0.5
char grade;    // a single character:       'A', '?'
bool passed;   // true or false
```

- The **type** decides what a variable can hold and what you can do with it.
- These four will carry us a long way. Text (strings) comes in Chapter 04.

3.4 Naming variables

The **rules** (break them and it won't compile):

- Letters, digits and underscores only, and the first character can't be a digit: `total2` is fine, `2total` is not.
- Names are case-sensitive: `width`, `Width` and `WIDTH` are three different variables.
- You can't use a **keyword** that C++ reserves for itself, such as `int`, `return` or `double`.

The **conventions** (good habits):

- Choose names that say what the value means: `width`, not `w` or `x1`.
- In this course, we write variable and function names in `snake_case`: `max_speed`, `get_value_from_user`.

4 Assignment and initialization

4.1 Assignment

After a variable is defined, `=` gives it a value. This is called **assignment**, and `=` is the **assignment operator**.

Listing 4 `variable_assignment.cpp`

```cpp
#include <iostream>

int main() {
    int width;                  // define a variable named width
    width = 5;                  // assign the value 5 to width
    std::cout << width << '\n'; // prints 5

    width = 7;                  // assign a new value: 5 is gone
    std::cout << width << '\n'; // prints 7
    return 0;
}
```

```
5
7
```

Assigning a new value **replaces** the old one.

4.2 Initialization

Defining and then assigning takes two statements. **Initialization** does both at once: it gives a variable its first value at the moment it is defined.

Listing 5 `variable_initialization.cpp`

```cpp
#include <iostream>

int main() {
    int width{5}; // define width AND give it the value 5
    std::cout << width << '\n';
    return 0;
}
```

```
5
```

- The value in the braces, `5`, is called the **initializer**.
- Initialization happens exactly once, when the variable is created. Assignment can happen as often as you like afterwards.

4.3 Five ways to initialize

```cpp
int a;     // default-initialization: no initializer (a holds garbage!)

// Traditional forms
int b = 5; // copy-initialization: value after an equals sign
int c(6);  // direct-initialization: value in parentheses

// Modern forms, using braces (preferred)
int d{7};  // direct-list-initialization: value in braces
int e{};   // value-initialization: empty braces give 0
```

- The forms with braces are called **list initialization** (or **brace initialization**).
- For a simple `int` like this, `b`, `c` and `d` all end up holding their value in the same way. But braces have one big advantage: next slide!

> Best practice
>
> Initialize with braces: `int d{7};` when you know the value, `int e{};` when you don't (yet).

4.4 Why braces? They stop narrowing

Listing 6 `narrowing.cpp`

```cpp
#include <iostream>

int main() {
    int a = 4.7; // copy-initialization: silently becomes 4
    std::cout << a << '\n';

    int b{4.7}; // list-initialization: compile error!
    return 0;
}
```

```
error: narrowing conversion of '4.7000000000000002e+0' from 'double' to 'int'
```

- `int a = 4.7;` compiles without a word, even with `-Wall -Wextra`, and quietly throws away the `.7`. That's a bug waiting to happen.
- `int b{4.7};` is a compile error. Brace initialization refuses to lose information. (The strange `4.7000000000000002` is how the computer really stores 4.7. More on that another day.)

4.5 Unused variables

Listing 7 `unused_variable.cpp`

```cpp
int main() {
    int x{5}; // defined, but never used
    return 0;
}
```

```
$ g++ -std=c++20 -Wall unused_variable.cpp
warning: unused variable 'x' [-Wunused-variable]
```

- A variable that's defined but never used is almost always a mistake (or leftover code), so g++ warns you.
- It's a **warning**, not an error: the program still compiles.

5 Input and output with `iostream`

5.1 The `iostream` library

- The **input/output library** is the part of the C++ standard library that reads input and prints output. The *io* in *iostream* stands for input/output.
- To use it, a file needs `#include <iostream>` at the top.
- It gives us two ready-made variables:
  - `std::cout`, "character output", which prints text to the screen;
  - `std::cin`, "character input", which reads what the user types on the keyboard.

5.2 Printing with `std::cout`

Listing 8 `use_stdcout.cpp`

```cpp
#include <iostream> // for std::cout

int main() {
    std::cout << "Hello world!\n"; // print some text
    std::cout << 4 << '\n';        // print a number

    int x{5};
    std::cout << x << '\n'; // print the value of x

    // Chain several outputs in one statement
    std::cout << "I am " << "Khiem " << "Nguyen" << '\n';

    int planets{8};
    int suns{1};
    std::cout << planets << " planets and " << suns << " sun\n";
    return 0;
}
```

```
Hello world!
4
5
I am Khiem Nguyen
8 planets and 1 sun
```

`<<` can be chained to print several things in one statement.

5.3 New lines: `'\n'` or `std::endl`?

Listing 9 `use_stdnewline.cpp`

```cpp
#include <iostream> // for std::cout and std::endl

int main() {
    std::cout << "Hi!" << std::endl;          // newline + flush
    std::cout << "My name is Khiem." << '\n'; // newline
    std::cout << "Nice to meet you!\n";       // '\n' in text
    return 0;
}
```

```
Hi!
My name is Khiem.
Nice to meet you!
```

- Both `'\n'` and `std::endl` move the cursor to the next line.
- `std::endl` also **flushes** the output: it forces everything waiting to be printed out to the screen *right now*. Doing that after every line is slow.

> Best practice
>
> Use `'\n'` for new lines. You rarely need `std::endl`.

5.4 Reading input with `std::cin`

Listing 10 `use_stdcin.cpp`

```cpp
#include <iostream> // for std::cout and std::cin

int main() {
    std::cout << "Enter a number: "; // prompt the user

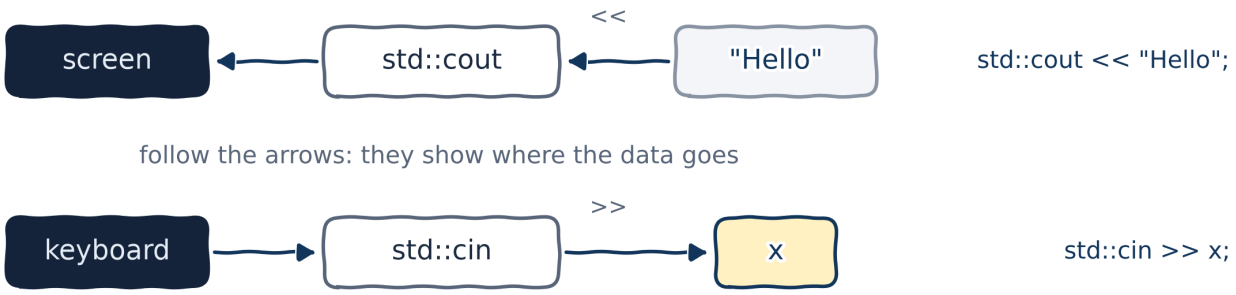
    int x{};       // a variable to hold the input
    std::cin >> x; // read a number into x

    std::cout << "You entered " << x << '\n';
    return 0;
}
```

```
$ ./use_stdcin
Enter a number: 7
You entered 7
```

- `std::cin >> x;` waits for the user to type something and press Enter, then stores it in `x`.
- We still initialize `x` with `{}`. If the read never happens (say, the input has already failed), `x` holds `0` instead of garbage. Always initializing is simply a good habit.

5.5 << and >>: follow the arrows



- `<<` is the **insertion operator**: `std::cout << 4` moves `4` *into* `std::cout`, and so to the screen.
- `>>` is the **extraction operator**: `std::cin >> x` moves what the user typed *out of* `std::cin` and into `x`.
- Mixing them up is a very common beginner's mistake. The arrows always point the way the data moves.

5.6 `std::cin` is buffered

Listing 11 `stdcin_buffered.cpp`

```cpp
#include <iostream>

int main() {
    std::cout << "Enter two numbers: ";

    int x{};
    std::cin >> x;
    std::cout << "You entered " << x << '\n';

    int y{};
    std::cin >> y;
    std::cout << "You entered " << y << '\n';
    return 0;
}
```

Run 1: type `4`, Enter, then `5`, Enter.

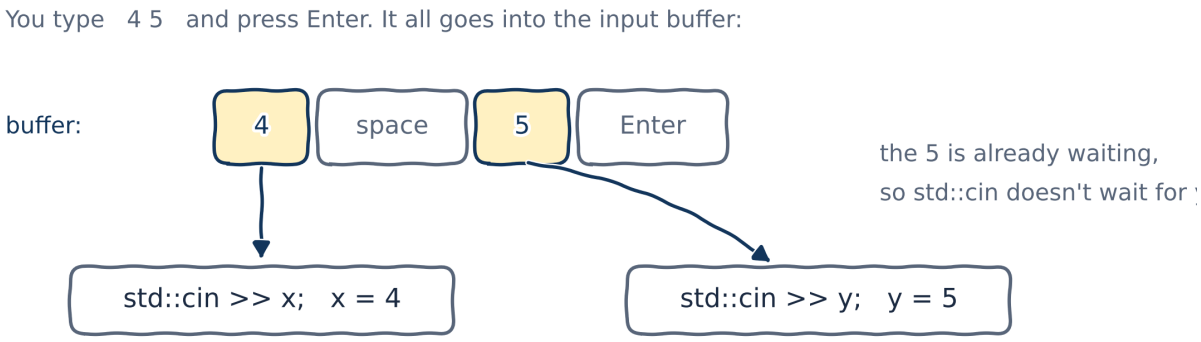
```
Enter two numbers: 4
You entered 4
5
You entered 5
```

Run 2: type `4 5`, then Enter.

```
Enter two numbers: 4 5
You entered 4
You entered 5
```

In run 2, the program never waits for the second number. Why?

5.7 How the input buffer works



- Everything you type goes into an **input buffer** when you press Enter.
- `>>` takes what it needs from the buffer and leaves the rest there. It only waits for you if the buffer is empty.
- That's why `4 5` on one line fills both `x` and `y`.

6 Uninitialized variables and undefined behaviour

6.1 Uninitialized variables

- C++ does **not** automatically set variables to zero. A variable that is never given a value holds whatever happened to be in that memory before: a **garbage value**.
- Such a variable is called **uninitialized**.

	The variable gets a known value…
Initialized	when it is defined: `int x{5};`
Assigned	after it is defined: `int x; x = 5;`
Uninitialized	not yet: `int x;`

"Initialized" and "uninitialized" are not quite opposites: a variable that is assigned a value later stops being uninitialized, even though it was never initialized.

6.2 What does an uninitialized variable hold?

Listing 12 `uninitialized_variable_01.cpp`

```cpp
#include <iostream>

int main() {
    int x;                  // uninitialized: no value given
    std::cout << x << '\n'; // who knows what we get?

    int y{}; // value-initialized to 0
    std::cout << y << '\n';

    int z{42}; // initialized to 42
    std::cout << z << '\n';
    return 0;
}
```

```
$ g++ -std=c++20 -Wall ...
warning: 'x' is used
uninitialized [-Wuninitialized]
$ ./a.out
0
0
42
```

On my machine, `x` happened to print `0`. Don't be fooled: nothing guarantees that. Look what happens in the next program.

6.3 Garbage values in action

Listing 13 `uninitialized_variable_02.cpp`

```cpp
#include <iostream>

void dirty_the_memory() {
    int a{42}; // these values are left behind in memory...
    int b{24};
    int c{66};
    std::cout << "Leaving behind " << a << ", " << b << ", " << c << '\n';
}

void see_the_junk() {
    int x; // uninitialized: x may land where a, b or c were
    std::cout << "Uninitialized x holds " << x << '\n';
}

int main() {
    dirty_the_memory();
    see_the_junk();
    return 0;
}
```

(Don't worry about the two extra functions: they're the topic of Chapter 02.) `dirty_the_memory()` leaves some values behind in memory; `see_the_junk()` then prints an uninitialized `x`. What will it print?

6.4 Same code, different answers

```
$ g++ -std=c++20 uninitialized_variable_02.cpp
$ ./a.out
Leaving behind 42, 24, 66
Uninitialized x holds 66
```

```
$ g++ -std=c++20 -O2 uninitialized_variable_02.cpp
$ ./a.out
Leaving behind 42, 24, 66
Uninitialized x holds 0
```

- Without optimization, `x` landed exactly where `c` had been, and picked up its leftover `66`.
- With `-O2` (asking the compiler to optimize), the same code printed `0`.
- Same code, different compiler setting, different answer. And neither answer is "wrong", because the program never said what `x` should be.

6.5 Undefined behaviour

Using an uninitialized variable is our first example of **undefined behaviour** (UB): code whose result the C++ language doesn't define. A program with UB might:

- print a different value every time you run it, or the same value every time;
- work on your computer and fail on someone else's, or after a change of compiler settings;
- crash, now or later;
- seem to work perfectly… until the day it doesn't.

> ! Key idea
>
> Avoid undefined behaviour at all costs. Here that's easy: **always initialize your variables**, and keep `-Wall -Wextra` on so the compiler can warn you.

7 Wrapping up

7.1 Summary

- Every program has a `main()` function, where it starts. Its statements run top to bottom, and most end with `;`.
- `g++` compiles source code into a program; syntax errors stop it. Read the first error first.
- Comments start with `//` or sit between `/*` and `*/`. Comment *why*, not *what*.
- A variable is a named object: a piece of memory holding a value of some type.
- Initialize variables with braces: `int x{5};` or `int x{};`. Braces also catch narrowing.
- `std::cout <<` prints, `std::cin >>` reads: the arrows show where the data goes. Use `'\n'` for new lines.
- Uninitialized variables give undefined behaviour. Always initialize!

7.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Keywords and naming identifiers
2. Whitespace and basic formatting
3. Introduction to literals and operators
4. Introduction to expressions

Next up, Chapter 02: functions and files. We'll stop repeating ourselves and split programs across several files.